

01_mysql_start_crud_beginner

Семинар: Сбор данных из различных источников. Основы работы с SQL

Файл 1. Запуск MySQL и базовые операции с таблицей `users`

Этот материал предназначен для слушателей с начальным уровнем подготовки. Мы будем двигаться постепенно: сначала запустим MySQL, затем создадим базу данных, таблицу пользователей и выполним базовые SQL-команды.

0. Что мы изучим в этом файле

После прохождения материала слушатель сможет:

- понять, что такое база данных, таблица, строка и столбец;
- запустить MySQL через Docker Compose;
- подключиться к MySQL из терминала;
- создать базу данных;
- создать таблицу `users`;
- добавить данные в таблицу;
- выбирать данные через `SELECT`;
- фильтровать строки через `WHERE`;
- сортировать данные через `ORDER BY`;
- считать простые показатели через `COUNT`, `AVG`, `SUM`, `MIN`, `MAX`;
- безопасно изменять и удалять строки через `UPDATE` и `DELETE`.

1. Простая аналогия: SQL-таблица как Excel

Перед началом договоримся о базовых понятиях.

Понятие	Простое объяснение	Аналогия с Excel
База данных	Место, где хранятся таблицы	Файл Excel
Таблица	Набор данных по одной теме	Лист Excel
Столбец	Один признак объекта	Колонка Excel
Строка	Одна запись	Строка Excel
SQL	Язык команд для работы с базой	Формулы и фильтры, но в виде команд

В нашем примере будет база данных интернет-магазина. В первой части мы создадим таблицу пользователей `users`.

2. Запуск MySQL через Docker Compose

2.1. Что такое Docker в этом занятии

Docker позволяет запустить MySQL в отдельном контейнере. Для слушателей это удобно: не нужно вручную устанавливать MySQL в систему, настраивать сервисы и искать конфигурационные файлы.

Простая схема:

```
Ваш компьютер
|
| запускает
v
Docker-контейнер
|
| внутри контейнера работает
v
MySQL Server
```

3. Файл `docker-compose.yml`

Создайте файл `docker-compose.yml` в рабочей папке проекта.

```
services:
  mysql:
    image: mysql:8.0
    container_name: mysql_container
    ports:
      - "3306:3306"
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: testdb
      MYSQL_USER: testuser
      MYSQL_PASSWORD: testpass
    volumes:
      - mysql_data:/var/lib/mysql
    restart: unless-stopped

volumes:
  mysql_data:
```

Разбор файла

Строка	Что означает
<code>services</code>	Список сервисов, которые нужно запустить
<code>mysql</code>	Название сервиса
<code>image: mysql:8.0</code>	Используем готовый образ MySQL версии 8.0
<code>container_name</code>	Имя контейнера, чтобы к нему было проще обращаться
<code>ports: "3306:3306"</code>	Открываем порт MySQL для подключения с компьютера
<code>MYSQL_ROOT_PASSWORD</code>	Пароль администратора MySQL
<code>MYSQL_DATABASE</code>	База, которая создается автоматически при первом запуске
<code>MYSQL_USER</code>	Пользователь, который создается автоматически
<code>MYSQL_PASSWORD</code>	Пароль этого пользователя
<code>volumes</code>	Постоянное хранилище данных MySQL
<code>restart: unless-stopped</code>	Контейнер будет перезапускаться, если его явно не остановили

Почему здесь используется `mysql_data`, а не `./mysql:/var/lib/mysql`

В исходном варианте использовалась папка `./mysql` и команда:

```
chmod 777 ./mysql
```

Для учебного занятия это может создать лишние сложности с правами доступа. Поэтому в обновлённом варианте используется именованный Docker volume `mysql_data`. Это проще для начинающих: Docker сам управляет папкой, где физически лежат данные MySQL.

4. Запуск контейнера

В терминале перейдите в папку, где лежит `docker-compose.yml`, и выполните:

```
docker compose up -d
```

Что делает команда:

- `docker compose` — запускает сервисы из файла `docker-compose.yml`;
- `up` — поднять сервисы;
- `-d` — запустить в фоновом режиме, чтобы терминал остался свободным.

Проверим, что контейнер работает:

```
docker ps
```

В списке должен быть контейнер `mysql_container`.

5. Подключение к MySQL

Выполните команду:

```
docker exec -it mysql_container mysql -u root -p
```

После этого MySQL попросит пароль.

Введите:

```
root
```

Пароль при вводе может не отображаться. Это нормально.

Если всё получилось, вы увидите приглашение MySQL:

```
mysql>
```

Теперь команды нужно писать уже не в обычный терминал, а внутри MySQL.

6. Проверка подключения к порту MySQL

Эта команда выполняется в обычном терминале, не внутри MySQL:

```
nc -zv 127.0.0.1 3306
```

Если соединение успешно, значит MySQL доступен на локальном компьютере через порт `3306`.

Часть 1. Создание базы данных

7. Посмотреть список баз данных

Команда:

```
SHOW DATABASES;
```

Эта команда показывает все базы данных, которые сейчас есть в MySQL.

Пример результата:

```
+-----+
| Database          |
+-----+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| testdb             |
+-----+
```

Служебные базы `information_schema`, `mysql`, `performance_schema`, `sys` создаются самим MySQL. Их пока не трогаем.

8. Создать новую базу данных

Создадим базу для занятия:

```
CREATE DATABASE my_database;
```

Если база создалась успешно, MySQL ответит примерно так:

```
Query OK, 1 row affected
```

Это означает, что команда выполнена без ошибки.

Более безопасный вариант

Если вы запускаете команду повторно, база уже может существовать. Тогда лучше использовать:

```
CREATE DATABASE IF NOT EXISTS my_database;
```

Фраза `IF NOT EXISTS` означает: «создай базу, если её ещё нет».

9. Выбрать активную базу данных

Создать базу недостаточно. Нужно ещё сказать MySQL, что дальше мы хотим работать именно с ней.

```
USE my_database;
```

Ожидаемый ответ:

Database changed

Теперь все следующие команды будут выполняться внутри базы `my_database`.

10. Посмотреть таблицы в текущей базе

```
SHOW TABLES;
```

Если база пустая, результат будет таким:

```
Empty set
```

Это не ошибка. Это значит, что таблиц пока нет.

Часть 2. Создание таблицы `users`

11. Бизнес-сценарий

Представим, что у нас есть интернет-магазин. Нам нужно хранить информацию о пользователях:

- уникальный номер пользователя;
- имя;
- email;
- возраст;
- страна;
- баланс счёта.

Для этого создадим таблицу `users`.

12. Создание таблицы

```
CREATE TABLE users (  
  id INT NOT NULL AUTO_INCREMENT,  
  name VARCHAR(50) NOT NULL,  
  email VARCHAR(100) NOT NULL,  
  age INT NOT NULL,  
  country VARCHAR(50) NOT NULL,  
  balance DECIMAL(10, 2) NOT NULL,  
  PRIMARY KEY (id)  
);
```

Разбор структуры таблицы

Поле	Тип	Что хранит
<code>id</code>	<code>INT</code>	Уникальный номер пользователя
<code>name</code>	<code>VARCHAR(50)</code>	Имя пользователя, максимум 50 символов
<code>email</code>	<code>VARCHAR(100)</code>	Email пользователя
<code>age</code>	<code>INT</code>	Возраст

Поле	Тип	Что хранит
country	VARCHAR(50)	Страна
balance	DECIMAL(10, 2)	Баланс с двумя знаками после запятой

Важные элементы команды

Элемент	Объяснение
CREATE TABLE users	Создать таблицу с именем users
NOT NULL	Поле обязательно должно быть заполнено
AUTO_INCREMENT	MySQL сам увеличивает номер id для новой строки
PRIMARY KEY (id)	id становится главным уникальным идентификатором строки
DECIMAL(10, 2)	Число до 10 цифр, из них 2 после запятой

Почему для денег используем DECIMAL, а не FLOAT

В исходном файле поле balance было задано как FLOAT. Для учебного примера это может работать, но для денежных значений лучше сразу использовать DECIMAL(10, 2).

Причина: DECIMAL хранит точные числовые значения, а FLOAT относится к приближительным типам. Для денег, цен и балансов точность важнее.

13. Проверить, что таблица создана

```
SHOW TABLES;
```

Ожидаемый результат:

```
+-----+
| Tables_in_my_database |
+-----+
| users                  |
+-----+
```

14. Посмотреть структуру таблицы

```
DESCRIBE users;
```

Ожидаемый результат:

```
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| id    | int           | NO   | PRI | NULL    | auto_increment |
| name  | varchar(50)   | NO   |     | NULL    |                |
| email | varchar(100)  | NO   |     | NULL    |                |
| age   | int           | NO   |     | NULL    |                |
| country | varchar(50)  | NO   |     | NULL    |                |
| balance | decimal(10,2) | NO   |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
```

Команда `DESCRIBE` полезна, когда нужно быстро вспомнить, какие поля есть в таблице.

Часть 3. Добавление данных

15. Добавить одного пользователя

```
INSERT INTO users (name, email, age, country, balance)
VALUES ('Ivan', 'ivan@example.com', 25, 'RUS', 100000.00);
```

Разбор:

Часть команды	Что означает
<code>INSERT INTO users</code>	Добавляем строку в таблицу <code>users</code>
<code>(name, email, age, country, balance)</code>	Указываем, какие поля заполняем
<code>VALUES (...)</code>	Передаём значения для этих полей

Мы не указываем `id`, потому что он создаётся автоматически через `AUTO_INCREMENT`.

16. Проверить содержимое таблицы

```
SELECT * FROM users;
```

Ожидаемый результат:

```
+----+-----+-----+-----+-----+-----+
| id | name | email           | age | country | balance |
+----+-----+-----+-----+-----+-----+
|  1 | Ivan | ivan@example.com |  25 |  RUS   | 100000.00 |
+----+-----+-----+-----+-----+-----+
```

`SELECT *` означает: «показать все столбцы».

17. Добавить несколько пользователей одной командой

```
INSERT INTO users (name, email, age, country, balance)
VALUES
  ('Gleb', 'gleb@example.com', 26, 'RUS', 90000.00),
  ('Sergey', 'sergey@example.com', 28, 'RUS', 90000.00),
  ('Andrew', 'andrew@example.com', 24, 'RUS', 95000.00),
  ('Bob', 'bob@example.com', 25, 'USA', 100000.00),
  ('Tom', 'tom@example.com', 29, 'USA', 110000.00);
```

Обратите внимание: в исходном файле у `Gleb` был email `gleb@exmple.com`. Здесь он исправлен на `gleb@example.com`, чтобы не закреплять опечатку в учебном материале.

18. Проверить все данные

```
SELECT * FROM users;
```

Ожидаемый результат:

id	name	email	age	country	balance
1	Ivan	ivan@example.com	25	RUS	100000.00
2	Gleb	gleb@example.com	26	RUS	90000.00
3	Sergey	sergey@example.com	28	RUS	90000.00
4	Andrew	andrew@example.com	24	RUS	95000.00
5	Bob	bob@example.com	25	USA	100000.00
6	Tom	tom@example.com	29	USA	110000.00

Часть 4. Выбор данных через `SELECT`

19. Выбрать все столбцы

```
SELECT * FROM users;
```

Эта команда удобна для знакомства с таблицей. Но в реальной работе лучше выбирать только нужные столбцы, особенно если таблица большая.

20. Выбрать конкретные столбцы

```
SELECT id, name, age
FROM users;
```

Ожидаемый результат:

id	name	age
1	Ivan	25
2	Gleb	26
3	Sergey	28
4	Andrew	24
5	Bob	25
6	Tom	29

Разбор:

- `SELECT id, name, age` — какие столбцы хотим получить;
- `FROM users` — из какой таблицы берём данные.

21. Мини-задание 1

Выведите только столбцы:

- `name`;
- `email`;
- `country`.

Ожидаемый SQL-шаблон:

```
SELECT ...  
FROM users;
```

Часть 5. Фильтрация через `WHERE`

22. Найти пользователей старше 26 лет

```
SELECT *  
FROM users  
WHERE age > 26;
```

Ожидаемый результат:

id	name	email	age	country	balance
3	Sergey	sergey@example.com	28	RUS	90000.00
6	Tom	tom@example.com	29	USA	110000.00

`WHERE` задаёт условие отбора строк.

23. Использовать несколько условий через `AND`

```
SELECT *  
FROM users  
WHERE age > 26 AND country = 'RUS';
```

Ожидаемый результат:

id	name	email	age	country	balance
3	Sergey	sergey@example.com	28	RUS	90000.00

`AND` означает, что должны выполняться оба условия одновременно:

- возраст больше 26;
- страна равна `RUS`.

24. Фильтрация по диапазону через `BETWEEN`

```
SELECT *
FROM users
WHERE balance BETWEEN 95000 AND 110000;
```

Ожидаемый результат:

id	name	email	age	country	balance
1	Ivan	ivan@example.com	25	RUS	100000.00
4	Andrew	andrew@example.com	24	RUS	95000.00
5	Bob	bob@example.com	25	USA	100000.00
6	Tom	tom@example.com	29	USA	110000.00

`BETWEEN 95000 AND 110000` означает: значение от 95000 до 110000 включительно.

25. Мини-задание 2

Напишите запросы:

1. Найти пользователей из страны `USA`.
2. Найти пользователей младше 26 лет.
3. Найти пользователей из `RUS` с балансом больше `95000`.

Часть 6. Сортировка через `ORDER BY`

26. Отсортировать пользователей по балансу по возрастанию

```
SELECT *
FROM users
ORDER BY balance;
```

По умолчанию `ORDER BY` сортирует по возрастанию: от меньшего значения к большему.

27. Отсортировать пользователей по балансу по убыванию

```
SELECT *
FROM users
ORDER BY balance DESC;
```

`DESC` означает убывание: от большего значения к меньшему.

Противоположный вариант — `ASC`, сортировка по возрастанию:

```
SELECT *
FROM users
ORDER BY balance ASC;
```

28. Сортировка по нескольким столбцам

```
SELECT *  
FROM users  
ORDER BY country ASC, balance DESC;
```

Такой запрос сначала сортирует пользователей по стране, а внутри каждой страны — по балансу от большего к меньшему.

29. Мини-задание 3

Напишите запросы:

1. Отсортировать пользователей по возрасту от младших к старшим.
2. Отсортировать пользователей по возрасту от старших к младшим.
3. Отсортировать пользователей по стране и имени.

Часть 7. Агрегатные функции

Агрегатные функции считают один итоговый показатель по набору строк.

Функция	Что делает
COUNT()	Считает количество строк
AVG()	Считает среднее значение
SUM()	Считает сумму
MAX()	Находит максимум
MIN()	Находит минимум

30. Посчитать пользователей из России

```
SELECT COUNT(*) AS rus_users_count  
FROM users  
WHERE country = 'RUS';
```

Ожидаемый результат:

```
+-----+  
| rus_users_count |  
+-----+  
|                4 |  
+-----+
```

`AS rus_users_count` задаёт понятное имя столбца в результате.

31. Посчитать пользователей из России с балансом 90000

```
SELECT COUNT(*) AS users_count
FROM users
WHERE country = 'RUS' AND balance = 90000;
```

Ожидаемый результат:

```
+-----+
| users_count |
+-----+
|          2 |
+-----+
```

32. Посчитать средний баланс

```
SELECT AVG(balance) AS avg_balance
FROM users;
```

Ожидаемый результат:

```
+-----+
| avg_balance |
+-----+
| 97500.0000 |
+-----+
```

Средний баланс считается так:

```
сумма всех балансов / количество пользователей
```

33. Посчитать сумму всех балансов

```
SELECT SUM(balance) AS total_balance
FROM users;
```

Ожидаемый результат:

```
+-----+
| total_balance |
+-----+
| 585000.00 |
+-----+
```

34. Найти максимальный и минимальный баланс

```
SELECT MAX(balance) AS max_balance
FROM users;
```

```
SELECT MIN(balance) AS min_balance
FROM users;
```

Можно объединить в один запрос:

```
SELECT
    MIN(balance) AS min_balance,
    MAX(balance) AS max_balance,
    AVG(balance) AS avg_balance,
    SUM(balance) AS total_balance
FROM users;
```

35. Мини-задание 4

Напишите запросы:

1. Сколько всего пользователей в таблице?
2. Какой средний возраст пользователей?
3. Какой максимальный возраст?
4. Какой минимальный баланс среди пользователей из `RUS` ?

Часть 8. Изменение данных через `UPDATE`

36. Главное правило безопасности

Перед изменением данных всегда сначала делаем проверочный `SELECT`.

Шаблон безопасной работы:

```
SELECT *
FROM users
WHERE id = 2;
```

Если результат правильный, только потом выполняем `UPDATE`:

```
UPDATE users
SET balance = 110000
WHERE id = 2;
```

После этого снова проверяем:

```
SELECT *
FROM users
WHERE id = 2;
```

37. Обновить баланс одного пользователя

```
UPDATE users
SET balance = 110000
```

```
WHERE id = 2;
```

Разбор:

Часть команды	Что означает
UPDATE users	Обновляем таблицу users
SET balance = 110000	Устанавливаем новое значение баланса
WHERE id = 2	Меняем только строку, где id = 2

38. Опасный пример: UPDATE без WHERE

Ниже пример, который нельзя выполнять без понимания последствий:

```
UPDATE users  
SET name = 'Ivan';
```

Эта команда изменит имя у всех пользователей в таблице.

Почему это опасно:

- нет условия WHERE;
- MySQL не знает, какую конкретную строку нужно изменить;
- поэтому команда применится ко всем строкам.

Правильный вариант для изменения одного пользователя:

```
UPDATE users  
SET name = 'Gleb'  
WHERE id = 2;
```

Запомните правило:

Нет WHERE в UPDATE — почти всегда риск испортить всю таблицу.

39. Восстановить имена после ошибочного обновления

Если на занятии вы специально показали опасный пример и все имена стали Ivan, можно восстановить данные так:

```
UPDATE users SET name = 'Gleb' WHERE id = 2;  
UPDATE users SET name = 'Sergey' WHERE id = 3;  
UPDATE users SET name = 'Andrew' WHERE id = 4;  
UPDATE users SET name = 'Bob' WHERE id = 5;  
UPDATE users SET name = 'Tom' WHERE id = 6;
```

После восстановления проверяем таблицу:

```
SELECT *  
FROM users;
```

40. Мини-задание 5

1. Найдите пользователя с `id = 3`.
2. Измените его баланс на `97000`.
3. Проверьте результат через `SELECT`.
4. Объясните, зачем в запросе нужен `WHERE`.

Часть 9. Удаление данных через `DELETE`

41. Безопасное удаление строки

Перед удалением тоже сначала делаем проверочный `SELECT`.

```
SELECT *  
FROM users  
WHERE id = 5;
```

Если это действительно нужная строка, выполняем удаление:

```
DELETE FROM users  
WHERE id = 5;
```

После удаления проверяем таблицу:

```
SELECT *  
FROM users;
```

42. Опасный пример: `DELETE` без `WHERE`

Нельзя бездумно выполнять:

```
DELETE FROM users;
```

Эта команда удалит все строки из таблицы `users`.

Правило:

```
Нет WHERE в DELETE — можно удалить всю таблицу данных.
```

43. Мини-задание 6

1. Добавьте нового пользователя:

```
INSERT INTO users (name, email, age, country, balance)  
VALUES ('TestUser', 'test@example.com', 30, 'RUS', 50000.00);
```

2. Найдите его через `SELECT`.
3. Удалите только этого пользователя.
4. Проверьте, что остальные пользователи остались в таблице.

Часть 10. Итоговая практика

44. Задания для самостоятельной работы

Выполните задания последовательно.

Задание 1

Выведите всю таблицу `users`.

Задание 2

Выведите только имена и email пользователей.

Задание 3

Найдите всех пользователей из `RUS`.

Задание 4

Найдите всех пользователей старше 25 лет.

Задание 5

Найдите пользователей с балансом от `90000` до `100000`.

Задание 6

Отсортируйте пользователей по балансу от большего к меньшему.

Задание 7

Посчитайте количество пользователей из `USA`.

Задание 8

Посчитайте средний баланс пользователей из `RUS`.

Задание 9

Измените баланс пользователя с `id = 1` на `120000`.

Задание 10

Удалите тестового пользователя, если он есть в таблице.

Часть 11. Шпаргалка SQL-команд

Задача	Команда
Показать базы данных	<code>SHOW DATABASES;</code>
Создать базу	<code>CREATE DATABASE my_database;</code>
Выбрать базу	<code>USE my_database;</code>
Показать таблицы	<code>SHOW TABLES;</code>
Создать таблицу	<code>CREATE TABLE ...;</code>
Посмотреть структуру таблицы	<code>DESCRIBE users;</code>
Добавить строку	<code>INSERT INTO ... VALUES ...;</code>

Задача	Команда
Выбрать все данные	<code>SELECT * FROM users;</code>
Выбрать отдельные столбцы	<code>SELECT name, age FROM users;</code>
Отфильтровать строки	<code>WHERE</code>
Отсортировать строки	<code>ORDER BY</code>
Посчитать строки	<code>COUNT(*)</code>
Посчитать среднее	<code>AVG(column)</code>
Посчитать сумму	<code>SUM(column)</code>
Найти максимум	<code>MAX(column)</code>
Найти минимум	<code>MIN(column)</code>
Обновить строку	<code>UPDATE ... SET ... WHERE ...;</code>
Удалить строку	<code>DELETE FROM ... WHERE ...;</code>

Часть 12. Типичные ошибки новичков

Ошибка	Почему возникает	Как исправить
Забыли <code>;</code>	MySQL ждёт продолжения команды	Добавить <code>;</code> и нажать Enter
Не выбрали базу через <code>USE</code>	MySQL не знает, где создавать таблицу	Выполнить <code>USE my_database;</code>
Ошибка в имени таблицы	Написали <code>user</code> вместо <code>users</code>	Проверить через <code>SHOW TABLES;</code>
Ошибка в имени столбца	Написали <code>balanse</code> вместо <code>balance</code>	Проверить через <code>DESCRIBE users;</code>
<code>UPDATE</code> изменил все строки	Не было <code>WHERE</code>	Сначала делать <code>SELECT</code> , потом <code>UPDATE</code>
<code>DELETE</code> удалил лишнее	Не было точного условия	Сначала делать <code>SELECT</code> , потом <code>DELETE</code>
Путают строки и столбцы	Нет визуального понимания таблицы	Сравнить с Excel: строки — записи, столбцы — признаки

Часть 13. Контрольные вопросы для преподавателя

Эти вопросы можно задать группе устно.

1. Чем база данных отличается от таблицы?
2. Что делает команда `USE my_database;`?
3. Зачем нужен `PRIMARY KEY`?
4. Почему поле `id` не нужно указывать при `INSERT`?
5. Чем `SELECT *` отличается от `SELECT name, age`?
6. Для чего используется `WHERE`?
7. Чем `ORDER BY balance` отличается от `ORDER BY balance DESC`?
8. Что делает `COUNT(*)`?
9. Почему опасно выполнять `UPDATE` без `WHERE`?
10. Почему опасно выполнять `DELETE` без `WHERE`?

Часть 14. Итог занятия по файлу

В этом файле мы прошли базовый цикл работы с SQL:

```
создали базу
-> создали таблицу
-> добавили данные
-> выбрали данные
-> отфильтровали данные
-> отсортировали данные
-> посчитали показатели
-> изменили данные
-> удалили данные
```

Это фундамент, который нужен перед переходом к следующему файлу. В следующей части мы добавим вторую таблицу `orders` и научимся связывать пользователей с заказами через `JOIN`.